

UNDEAD LEGION

MODULAR SKELETON ARMY

DOCUMENTATION · UNITY PACKAGE

UNITY 6 (6000.4) · UNIVERSAL RENDER PIPELINE

CONTENTS

- | | |
|---------------------------|-------------------------------|
| 1. Overview | 8. The Animator controller |
| 2. Requirements and setup | 9. Root motion and speeds |
| 3. Folder structure | 10. Scripts |
| 4. Characters | 11. The demo scene |
| 5. Armor modules | 12. Using your own animations |
| 6. Weapons | 13. Technical details |
| 7. Animations | 14. Support |

1. OVERVIEW

Undead Legion is a skeleton army built on **one shared humanoid rig**. Six class presets, every armor piece and every animation use the same skeleton, so any armor piece can be worn by any skeleton and every animation plays on every character.

- **6 skeleton classes:** Knight, Warrior, Archer, Assassin, Mage, Necromancer, each a ready prefab.
- **48 armor modules** as separate prefabs: helms, chests, gloves, greaves, boots, skirts, robes and pants.
- **12 weapon prefabs** with adjustable grip points, including a skinned recurve bow whose string follows the draw hand.
- **58 animation clips** plus grip pose data: idles, two locomotion sets with root motion, one- and two-handed attacks, a held block, a bow shot, spell casts, class specials, two hit and two stagger reactions, four deaths, additive twitches and empty-hand finger idles.
- A **Humanoid avatar** on every character, so the clips also retarget to other humanoids and other humanoid clips play on the skeletons.
- A **demo scene** to browse every character, module, weapon and clip.

2. REQUIREMENTS AND SETUP

- Unity 6 (6000.4 or newer) with the **Universal Render Pipeline** (17.4). The materials are URP/Lit.
- uGUI (included with Unity) for the demo interface. The demo reads input through either the Input System package or the legacy Input Manager.

Import the package, then open `UndeadLegion/Demo/Scenes/UndeadLegion_Demo.unity` and press Play. To use a character in your own scene, drag a prefab from `UndeadLegion/Prefabs/Characters` into it. Each prefab already carries its Animator with the shared controller, its default armor, and the weapon and armor components described below.

If the materials appear pink, the project is not using URP: install the Universal RP package and assign a URP asset in *Project Settings > Graphics* and *Quality*.

3. FOLDER STRUCTURE

FOLDER	CONTENTS
--------	----------

Animations	The clips (Skeleton@<Clip>.fbx), the controller AC_Skeleton and its avatar masks.
Demo	The demo scene and its scripts (namespace UndeadLegion.Demo).
Documentation	This file.
Materials/URP	A body and an armor material per character, a material per weapon, the demo floor.
Models/<Character>	SK_<Character>.fbx: the skeleton and its bare body, the avatar source.
Models/Armor/<Character>	SK_<Character>_<Module>.fbx: one skinned armor piece each.
Models/Weapons	SM_<Weapon>.fbx.
Prefabs/Characters	PF_<Character>: the dressed, ready-to-use characters.
Prefabs/Armor/<Character>	A_<Character>_<Module>: the armor modules.
Prefabs/Weapons	W_<Weapon>: the weapons, each with a child Grip.
Textures	Color, normal and packed metallic/smoothness maps per character and weapon.

4. CHARACTERS

Every character shares the same 68-bone skeleton (Mixamo-style bone names: Hips, Spine, Spine1, Spine2, Neck, Head, Jaw, LeftArm, LeftForeArm, LeftHand, ..., full fingers, and two weapon socket bones LeftWeaponSocket / RightWeaponSocket under the hands). Characters stand 1.7 m to the top of the skull, with the hips at 0.98 m.

PREFAB	DEFAULT ARMOR	BODY TRIS	ARMOR TRIS	TOTAL TRIS
PF_SkeletonKnight	Helm, Chest, Gloves, Greaves, Boots, Skirt	7,011	9,717	16,728
PF_SkeletonWarrior	Chest, Gloves, Greaves, Boots, Skirt	7,588	9,391	16,979
PF_SkeletonArcher	Helm, Chest, Gloves, Boots, Skirt	6,364	10,992	17,356
PF_SkeletonAssassin	Helm, Chest, Gloves, Pants, Boots, Skirt	7,588	11,282	18,870
PF_SkeletonMage	Helm, Robe, Gloves, Greaves, Boots	7,502	10,147	17,649
PF_SkeletonNecromancer	Helm, Robe, Gloves, Greaves, Boots	7,502	9,882	17,384

Each character has two materials, M_<Character>_Body and M_<Character>_Armor, with 1024×1024 color, normal and metallic/smoothness maps. Armor materials render both faces, so torn cloth and open hoods read correctly from behind.

5. ARMOR MODULES

Every armor piece is its own prefab in Prefabs/Armor/<Character>. A piece is skinned to the shared skeleton and binds to **any** character by bone name, so a Warrior can wear the Knight's helm or the Mage's robe.

SET	MODULES
Knight	Helm, Chest, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R, Skirt
Warrior	Chest, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R, Skirt
Archer	Helm, Chest, Glove_L, Glove_R, Boot_L, Boot_R, Skirt

Assassin	Helm, Chest, Glove_L, Glove_R, Pants, Boot_L, Boot_R, Skirt
Mage	Helm, Robe, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R
Necromancer	Helm, Robe, Glove_L, Glove_R, Greave_L, Greave_R, Boot_L, Boot_R

WEARING AND REMOVING PIECES

Each character prefab has a `SkeletonModules` component listing its default set. At runtime:

```
var modules = character.GetComponent<SkeletonModules>();
modules.Wear(knightHelmPrefab);    // attach any module prefab (bound by bone name)
modules.Remove(knightHelmPrefab);  // a default piece is switched off, a borrowed one destroyed
modules.Set(0, false);             // toggle a default piece by index
modules.SetAll(true);              // every default piece on
```

Without `SkeletonModules`, `ArmorModule.Wear(modulePrefab, characterRoot)` instantiates a piece and binds it to any character with the shared skeleton. When you swap a piece, take off the character's own piece of the same slot first (for example its Helm) so two pieces do not overlap.

A module prefab dropped into a scene on its own is a prop in its rest pose. It animates once it is attached to a character.

6. WEAPONS

PREFAB	WEAPON	TRIS	HELD
W_H1Sword	Sword	2,496	right hand
W_H1Axe	Axe	2,480	right hand
W_H1Mace	Mace	3,249	right hand
W_H1Dagger	Dagger	1,742	either hand
W_H1Wand	Wand	5,074	right hand
W_H1HeaterShield	Heater shield	718	left hand
W_H1RoundShield	Round shield	2,168	left hand
W_H2Longsword	Longsword	2,916	two-handed
W_H2Axe	Battle axe	3,334	two-handed
W_H2MagicStuff	Staff	5,074	right hand, both hands in the staff cast
W_H2Recurvebow	Recurve bow (skinned: the string bends with the draw)	3,869	left hand
W_Arrow	Arrow	498	right hand, with the bow

LOADOUTS

`SkeletonWeapon` on each character holds a list of **loadouts** (a display name and the items it puts in which hand): Sword + shield, Sword, Axe + round shield, Mace, Dagger, Two daggers, Longsword (2H), Battle axe (2H), Recurve bow, Staff, Wand.

```
var weapon = character.GetComponent<SkeletonWeapon>();  
weapon.Equip(0);           // equip loadout 0 (see weapon.Count / weapon.NameAt(i))  
weapon.Clear();           // empty hands
```

HAND SLOTS AND GRIPS

Two transforms meet when a weapon is equipped: the character's hand slot (**RightHandSlot** / **LeftHandSlot**, children of the weapon socket bones) and the weapon prefab's child **Grip**. The weapon is placed so its **Grip** matches the slot in position and rotation. Move or rotate either one with the gizmo to adjust how a weapon sits in the hand; all six characters share the skeleton, so one slot setup fits all. The **Grip** convention: +**Y** along the handle toward the blade, +**Z** out of the back of the hand.

When a hand holds an item its fingers close into a grip pose; an empty hand plays a relaxed finger idle (the **LeftFingers** / **RightFingers** layers, section 8).

THE BOW

Shoot_01 carries two animation events, **BowAttach** and **BowRelease**. Between them the bow's string follows the draw hand and the limbs bend (**BowString** on the bow prefab); on the release the string snaps back and **SkeletonWeapon** can launch the held arrow as a simple **ArrowProjectile** (the demo's shot; no collision or damage). Receivers are optional: the events use *DontRequireReceiver*.

7. ANIMATIONS

All clips are 30 fps Humanoid clips, `Skeleton@<Clip>.fbx` in Animations. Loops close seamlessly. Clips marked *root motion* move the character with *Apply Root Motion* on and play in place with it off; the others play in place.

GROUP	CLIPS	TYPE
Idles	Idle_01, Idle_02, Idle_03	loop
Combat idle	Idle_1H_Combat (one-handed weapons and shields: both hands ready)	loop
Weapon idles	Idle_TwoHanded, Idle_Bow, Idle_Staff	loop
Locomotion, heavy set	Walk_Fwd_01, Walk_Back_01, Run_Fwd_01, Strafe_Left_01, Strafe_Right_01	loop, root motion
Locomotion, upright set	Walk_Fwd_02, Walk_Back_02, Run_Fwd_02, Strafe_Left_02, Strafe_Right_02	loop, root motion
Right arm	Attack_R_01_Stab, Attack_R_02_Swing, Attack_R_03_Swing	one-shot
Left arm	Attack_L_01_Stab, Attack_L_02_Swing, Attack_L_03_Swing, Block_L_Idle (held block)	one-shot; the block loops
Two-handed	Attack_2H_01_Swing (overhead chop), Attack_2H_02_Swing (sweep)	one-shot
Bow	Shoot_01 (raise, draw, release, lower)	one-shot, events
Magic	Cast_Wand_01, Cast_Wand_02, Cast_Staff_01	one-shot
Specials	Taunt_01, Taunt_02, Taunt_03, Rally, Cutthroat, Summon, AOE_Cast	one-shot
Reactions	Hit_01 (additive: a head-and-shoulders recoil) and Hit_02 (additive: a gut blow, the chest folds), both added over whatever plays on the Hit layer; Stagger_01 / Stagger_02 (Hit_01 / Hit_02 while stepping back one walk cycle; 66 cm of root motion)	additive overlay / one-shot, in place
Deaths	Death_01 (drop), Death_02 (face-down), Death_03 (on the back), Death_04 (limp)	one-shot, ends lying
Twitches	Twitch_01, Twitch_02, Twitch_03 (head and jaw)	additive
Finger idles	Hand_Idle_L, Hand_Idle_R (relaxed empty hand)	loop, finger-only
Grip pose	Grip (finger pose data used for held items; not a playable state)	data

The one-handed attacks and casts carry the whole body, so they read complete when a character stands; on the move they play on the arm and upper-body layers over the legs of a walk or run (section 8).

8. THE ANIMATOR CONTROLLER

AC_Skeleton is shared by every character. Its layers:

LAYER	BLENDING	USE
Base	override	Every clip as a full-body state. Idle_01 is the default.

UpperBody	override, mask AM_UpperBody	Upper-body clips (wand casts, the shot, Rally) over a locomotion loop.
LeftArm / RightArm	override, masks AM_LeftArm / AM_RightArm	The per-arm attacks over a locomotion loop; Block_L_Idle held on LeftArm over anything.
Twitch	additive	One twitch on a trigger (Twitch , TwitchIndex), used by SkeletonTwitch .
TwitchLoop_01..03	additive, weight 0	A looping twitch each; raise the layer weight to keep one running.
LeftFingers / RightFingers	override, finger masks	The relaxed finger idles; SkeletonWeapon sets their weight (1 for an empty hand, 0 while holding).
Hit	additive, mask AM_UpperBody	Hit reactions added over whatever Base plays, the legs untouched: animator.CrossFadeInFixedTime("Hit_01", 0.05f, animator.GetLayerIndex("Hit"), 0f) (or Hit_02). Lower the layer weight for a lighter hit.

Every layer other than Base has an **Empty** default state. To play an arm attack while walking, cross-fade the attack on its arm layer and let Base keep the walk:

```
animator.CrossFadeInFixedTime("Attack_R_02_Swing", 0.15f, animator.GetLayerIndex("RightArm"));
```

Every **Attack_*** state, on Base and on the arm layers, plays at the float parameter **AttackSpeed** (default 1). **SkeletonWeapon** sets it from the equipped loadout, so each weapon keeps its own pace wherever its attacks play: the sword's attacks run 30 % faster, the dagger's 70 % faster (both hands with two daggers), every other weapon as authored. The table is **SkeletonWeapon.AttackSpeedFor**; edit it to retune a weapon.

Two-handed attacks, staff casts, specials and deaths are full-body and belong on Base. **SkeletonGripState** on the two-handed and staff states closes the second hand on the weapon.

9. ROOT MOTION AND SPEEDS

Locomotion clips carry their travel as root motion. Measured on the characters (identical for all six, the avatar is shared); use these to match an in-place blend tree or an agent's speed without foot sliding:

CLIP	LENGTH	TRAVEL PER LOOP	SPEED
Walk_Fwd_01	1.400 s	0.724 m	0.52 m/s
Walk_Back_01	1.967 s	-0.662 m	0.34 m/s
Run_Fwd_01	0.967 s	1.107 m	1.14 m/s
Strafe_Left_01 / Strafe_Right_01	1.467 s	0.293 m sideways	0.20 m/s
Walk_Fwd_02	1.133 s	1.455 m	1.28 m/s
Walk_Back_02	1.467 s	-1.132 m	0.77 m/s
Run_Fwd_02	0.833 s	1.562 m	1.87 m/s
Strafe_Left_02 / Strafe_Right_02	1.300 s	0.843 m sideways	0.64 m/s

The **_01** set is a heavy, shambling gait for rank-and-file skeletons; the **_02** set is upright, for captains and casters. With *Apply Root Motion* off every clip plays in place, which suits NavMesh-driven movement.

10. SCRIPTS

All scripts are in `Demo/Scripts`, namespace `UndeadLegion.Demo`. The character-side ones are usable in a game as they are.

SCRIPT	ON	DOES
<code>SkeletonModules</code>	character	Toggles the default armor set; wears and removes any module prefab.
<code>ArmorModule</code> / <code>ArmorPiece</code>	armor prefab / attached piece	Binds a piece to a character by bone name; marks where an attached piece came from.
<code>SkeletonWeapon</code>	character	Loadouts, hand slots, the grip pose, finger idles, the attack speed per weapon, the bow events and the fired arrow.
<code>BowString</code>	bow prefab	Pulls the string to the draw hand and bends the limbs.
<code>ArrowProjectile</code>	spawned arrow	A straight flight that removes itself after its lifetime.
<code>SkeletonGripState</code>	Animator states	Closes the second hand on a two-handed weapon during the state.
<code>SkeletonFootLock</code>	character	Holds the feet still on their IK goals while a looping idle plays (needs IK Pass on the Base layer, already set).
<code>SkeletonEyes</code>	character	Two small glowing spheres in the eye sockets, seated on each skull's own socket and following the head; red by default. The color is HDR (values above 1 glow with Bloom); <code>SetColor()</code> and <code>SetVisible()</code> at runtime; <code>addLight</code> adds a faint light at the face (off by default).
<code>SkeletonTwitch</code>	character	Random additive head and jaw twitches, for a crowd that never quite stands still.
<code>SkeletonShowcase</code> , <code>DemoUI</code> , <code>DemoTurntable</code> , <code>DemoEventSystemBootstrap</code>	demo scene	The demo browser, its interface and camera.

11. THE DEMO SCENE

`UndeadLegion_Demo` lists the characters (top left), the armor modules of the current character and the sets to borrow from (left) and the animations by section (right). Click a clip to play it; the arm attacks go on their arm layer when a locomotion loop is running, and the block button holds the block until clicked again. The twitch and finger-idle entries are toggles. The scene's global Volume (`Demo_Volume`) adds Bloom above an HDR value of 1, so only the eyes glow. The footer switches the glowing eyes, root motion and the turntable and recenters the character. Drag to orbit the camera, scroll to zoom.

12. USING YOUR OWN ANIMATIONS

The characters use Unity's Humanoid avatar, so any Humanoid clip plays on them: set the clip's rig to Humanoid and add it to the controller. The pack's clips also drive the finger, jaw and extra spine bones; third-party clips leave those at rest. To use the pack's clips on another Humanoid character, assign that character's avatar to the Animator and use `AC_Skeleton` or your own controller.

13. TECHNICAL DETAILS

Skeleton	68 bones, Humanoid avatar with 53 mapped bones (fingers, jaw and upper chest included), two weapon socket bones
Scale	1 unit = 1 m; import scale 1; characters 1.70 m to the skull top
Skinning	up to 4 influences per vertex
Characters	16.7k–18.9k triangles fully dressed (table in section 4)
Textures	1024×1024 color, normal, metallic/smoothness (R = metallic, A = smoothness) for the characters, 512×512 for the weapons; every texture a power of two
Materials	URP/Lit; armor double-sided
Animations	58 clips + grip pose data, 30 fps, Humanoid

14. SUPPORT

For questions or issues, email vadymudod@hotmail.com. Please include your Unity version and a short description or screenshot of the problem.